Integrating DevSecOps Security Scanning with Grafana/Prometheus
Project Finalized

Yes. **Combining DevSecOps + Prometheus/Grafana can make a very strong IA-II project**, but I would frame it carefully so it doesn't look like you are simply repeating Lab 9.

Your project should be about **security-aware observability** rather than "monitoring + security" as two unrelated tools.

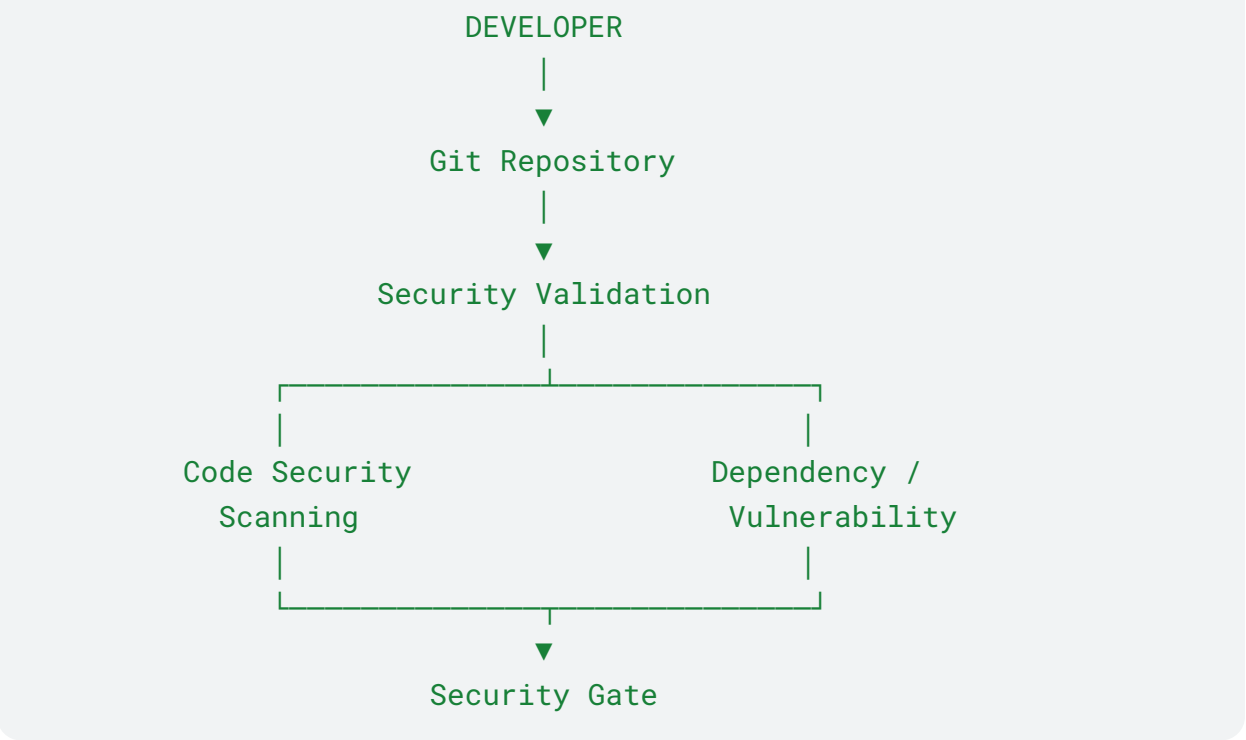
Recommended project

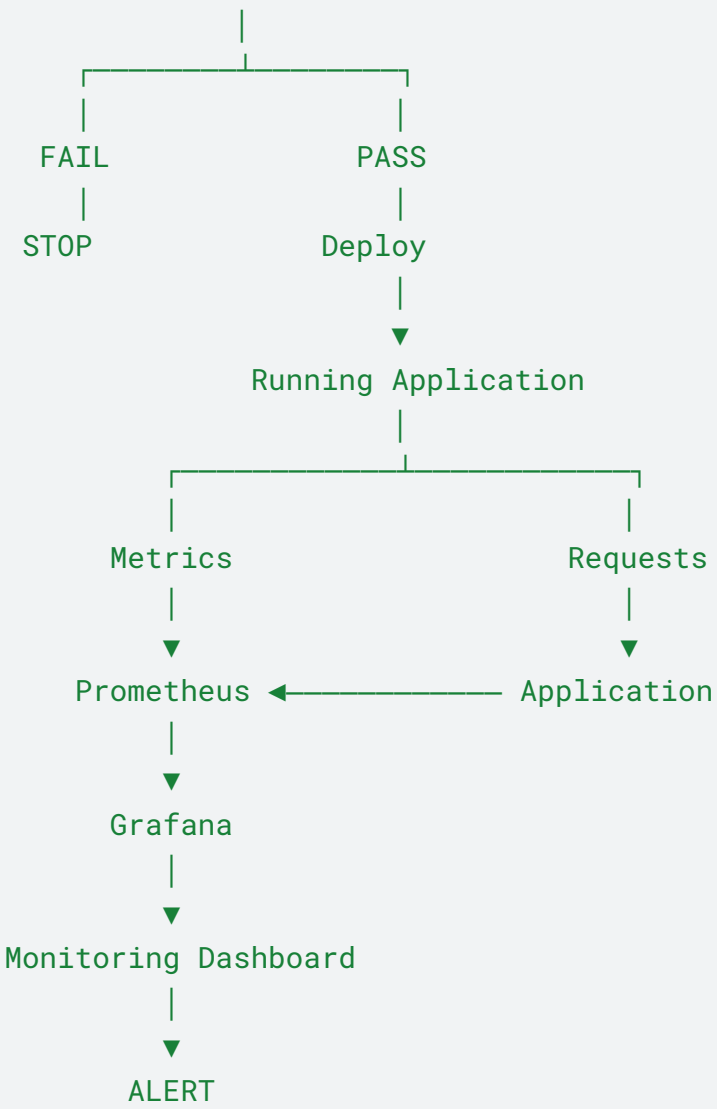
Secure & Observable DevOps Pipeline: Integrating DevSecOps Security Scanning with Real-Time Application Monitoring

Core idea

You build a small web application and create a miniature production-style environment:

None





This gives you **two distinct stories**:

DevSecOps:

> "Is the application safe enough to deploy?"

Observability:

> "Once deployed, is the application healthy?"

That's a much stronger project than simply showing two tools.

---

1. What exactly are we building?

I recommend a very small **Python Flask web application**.

It doesn't need to be complicated.

For example:

```
None
SecureMonitor/
│
├── app/
│   ├── app.py
│   ├── requirements.txt
│   └── templates/
│       └── index.html
│
├── tests/
│   └── test_app.py
│
├── monitoring/
│   ├── prometheus.yml
│   └── alerts.yml
│
├── security/
│   └── zap/
│
├── .github/
│   └── workflows/
│       └── security.yml
│
└── README.md
```

The application exposes:

```
None
/
```

and a metrics endpoint such as:

```
None
/metrics
```

Prometheus periodically scrapes metrics from monitored targets; this is its fundamental monitoring model. (Prometheus)

Grafana then connects to Prometheus as a data source and uses PromQL to query and visualize those time-series metrics. (Grafana Labs)

---

2. Phase 1 — Application Development

Goal

Create a simple application that can actually generate interesting metrics.

For example:

```
None
SecureMonitor App


┌──────────────────────────────┐
│                              │
│       SecureMonitor          │
│                              │
│   Welcome to our DevOps      │
│       demonstration          │
│                              │
│  [ Generate Traffic ]        │
│                              │
└──────────────────────────────┘
```

The application should have several endpoints:

```
/
 /health
 /api/data
 /api/error
 /metrics
```

Why multiple endpoints?

Because you want to generate different monitoring scenarios.

For example:

```
/              → successful requests
/health        → health check
/api/data      → normal API traffic
/api/error     → simulated failures
```

---

3. Phase 2 — Instrument the Application

This is where the project becomes much more interesting.

The application should produce metrics such as:

Request count

```
http_requests_total
```

Request latency

```
http_request_duration_seconds
```

Error count

```
None
http_requests_failed_total
```

Application health

```
None
app_health
```

You can therefore answer questions such as:

> How many requests is my application receiving?
>
> How many are failing?
>
> How long do requests take?
>
> Is the application currently healthy?

---

4. Phase 3 — Prometheus

Now introduce Prometheus.

Architecture:

```
None
                    Flask Application
                           |
                           | /metrics
                           ▼
                      Prometheus
                           |
                    Time-series DB
                           |
                           ▼
                        Grafana
```

Prometheus collects and stores metrics as time series. (Prometheus)

Your `prometheus.yml` will tell Prometheus where your application is.

Conceptually:

```
None
scrape_configs:
  - job_name: "secure-monitor"
    static_configs:
      - targets: ["localhost:5000"]
```

Then demonstrate that Prometheus can see the application.

---

5. Phase 4 — Grafana Dashboard

Now create the **main visual output of your project**.

Grafana has native support for Prometheus as a data source. (Grafana Labs)

Your dashboard should contain around **6 panels**.

Panel 1 — Application Status

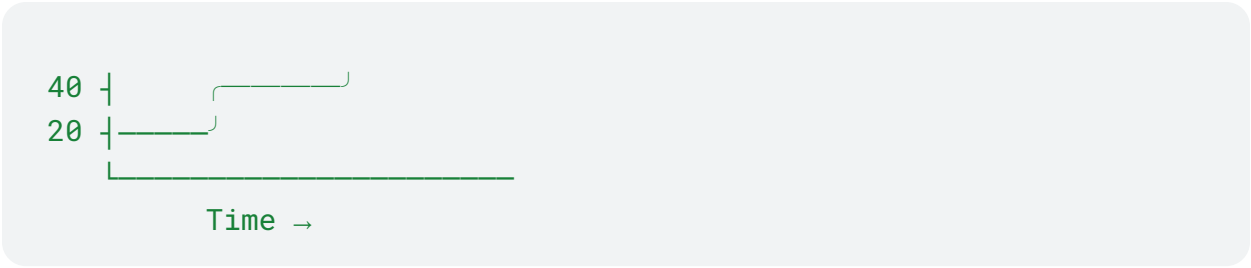
```
None
APPLICATION STATUS

        🟢 UP
```

Panel 2 — Request Rate

```
None
Requests / Second

100 ┤                    ╭─────
 80 ┤               ╭─────╯
 60 ┤          ╭─────╯
```



Panel 3 — Error Rate

None

```
ERROR RATE

0% ──────────────────────────
```

Normally low.

Then you'll intentionally generate errors.

Panel 4 — Response Time

None



Panel 5 — Total Requests

None

```
TOTAL REQUESTS

        12,483
```

Panel 6 — Security Status

This is where your two concepts visually come together:

```
None
SECURITY STATUS

       🟢 SECURE

Last Scan: PASSED
Critical: 0
High:     0
Medium:   1
```

That gives you a **single dashboard telling the story of the application**.

---

6. Phase 5 — Introduce DevSecOps

Now we move to the second half of the project.

Your fundamental question becomes:

> **"Can we automatically detect security problems before deployment?"**

Your pipeline becomes:

```
None
Code
 ↓
Security Scan
 ↓
Dependency Scan
 ↓
Security Decision
 ↓
Deploy
```

This is the **shift-left** idea:

```
None
Traditional:

Development → Deployment → Security

DevSecOps:

Development
     ↓
Security
     ↓
Testing
     ↓
Deployment
```

---

7. Phase 6 — Automated Security Scanning

I would use **two different security checks** rather than relying on a single scanner.

A. Code scanning

Use **CodeQL** for static code analysis.

GitHub's CodeQL can identify vulnerabilities and coding errors, with findings appearing as code-scanning alerts. (GitHub Docs)

You can demonstrate:

```
None
Code
 ↓
CodeQL
 ↓
```

```
Security Analysis
 ↓
Alerts
```

B. Web application scanning

Use **OWASP ZAP** against your running application.

ZAP's baseline scan is specifically designed to perform a relatively short passive scan and is suitable for automation/CI environments. (ZAP)

So:

```
None
Source Code
     |
     ▼
   CodeQL
     |
     ▼
Code vulnerabilities

Running Application
     |
     ▼
   OWASP ZAP
     |
     ▼
Web vulnerabilities
```

This is significantly more impressive than saying:

> "We used a security scanner."

---

8. Phase 7 — Create a Security Gate

This is the **most important part of the DevSecOps demonstration**.

Your pipeline should have a rule:

> **If a critical/high security issue is detected, deployment must not proceed.**

So:

```
None
Security Scan
      |
      ▼
  Vulnerability?
    /       \
  YES        NO
   |          |
   ▼          ▼
  FAIL       PASS
   |          |
   ▼          ▼
STOP       DEPLOY
```

This is where you demonstrate actual **DevSecOps**, rather than simply running a security scanner.

---

9. Phase 8 — Deliberately Introduce a Vulnerability

This will make your live demonstration much stronger.

Start with:

```
None
Application
     ↓
Security Scan
```

```
      ↓
✓ PASS
      ↓
Deployment
      ↓
Application running
```

Then introduce a **safe, controlled vulnerability** in your demo application.

Run the pipeline again:

```
None
Application
      ↓
Security Scan
      ↓
❌ Vulnerability detected
      ↓
Security Gate
      ↓
❌ DEPLOYMENT BLOCKED
```

This is your first major "wow" moment.

Then fix it.

```
None
Fix vulnerability
        ↓
Run scan again
        ↓
✓ Security scan passed
        ↓
Deployment allowed
        ↓
Application running
```

Do not use a real external target or exploit a real system. Keep all security testing confined to your own intentionally vulnerable local/demo application.

---

10. Phase 9 — Monitoring the Running Application

Once the application successfully deploys:

```
None
Application
     ↓
Metrics
     ↓
Prometheus
     ↓
Grafana
```

Now generate traffic.

For example:

```
None
Normal traffic
     ↓
100 requests
     ↓
Dashboard changes
```

You can show:

```
None
Requests/sec ↑
Total requests ↑
Latency changes
```

```
Error rate remains low
```

---

11. Phase 10 — Create a Monitoring Incident

This is your **second major "wow" moment**.

Your application normally looks like:

```
None
STATUS:        🟢 UP
ERROR RATE:    0.2%
LATENCY:       80 ms
REQUESTS:      20/sec
```

Now generate many failed requests.

Your dashboard changes:

```
None
STATUS:        🔴 DEGRADED
ERROR RATE:    18.7%
LATENCY:       950 ms
REQUESTS:      85/sec
```

Grafana's role is to query the Prometheus data and visualize it; it can also be used to create alert rules. (Grafana Labs)

---

12. Phase 11 — Add Alerting

Now go one step beyond simply displaying graphs.

Create a rule such as:

```
IF error rate > 5%
FOR 1 minute

THEN

ALERT:
"High Application Error Rate"
```

Your dashboard could display:

```
┌──────────────────────────────────────┐
│       🚨 APPLICATION ALERT           │
│                                      │
│  High Error Rate Detected            │
│                                      │
│  Current Error Rate: 18.7%           │
│  Threshold:          5%              │
└──────────────────────────────────────┘
```

This turns the project from:

> "We made a Grafana dashboard."

into:

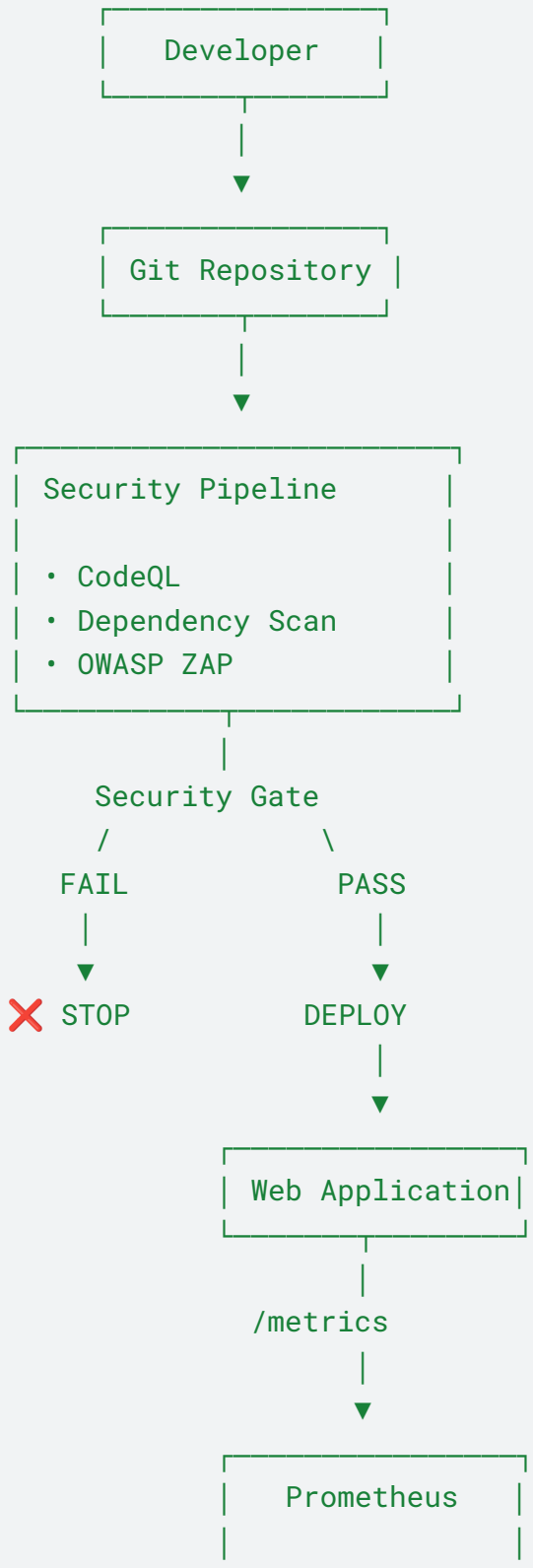
> **"We built an automated security and observability system."**

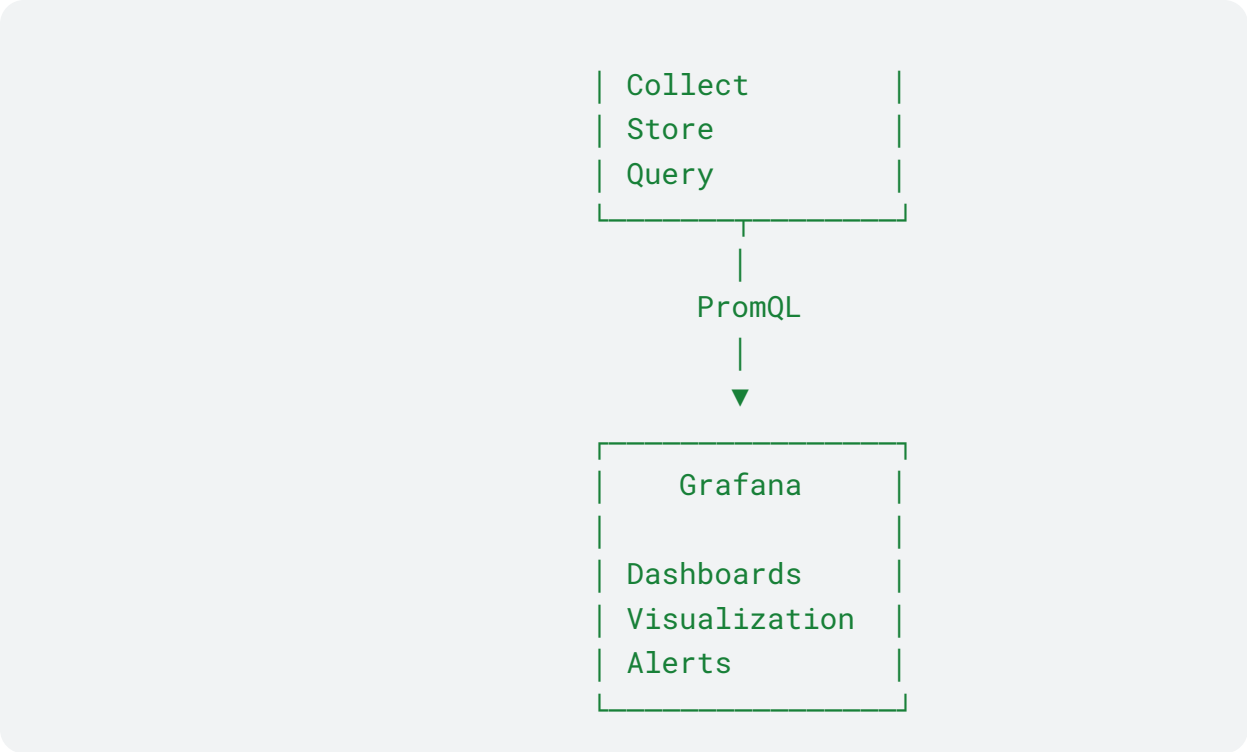
---

13. Final Architecture

This should be one of your most important presentation diagrams.

None


Developer
Git Repository
Security Pipeline
• CodeQL
• Dependency Scan
• OWASP ZAP
Security Gate
FAIL
PASS
❌ STOP
DEPLOY
Web Application
/metrics
Prometheus



---

14. What will actually be displayed at the end?

This is the most important question you asked.

Your **final deliverable should not be a collection of separate tools**.

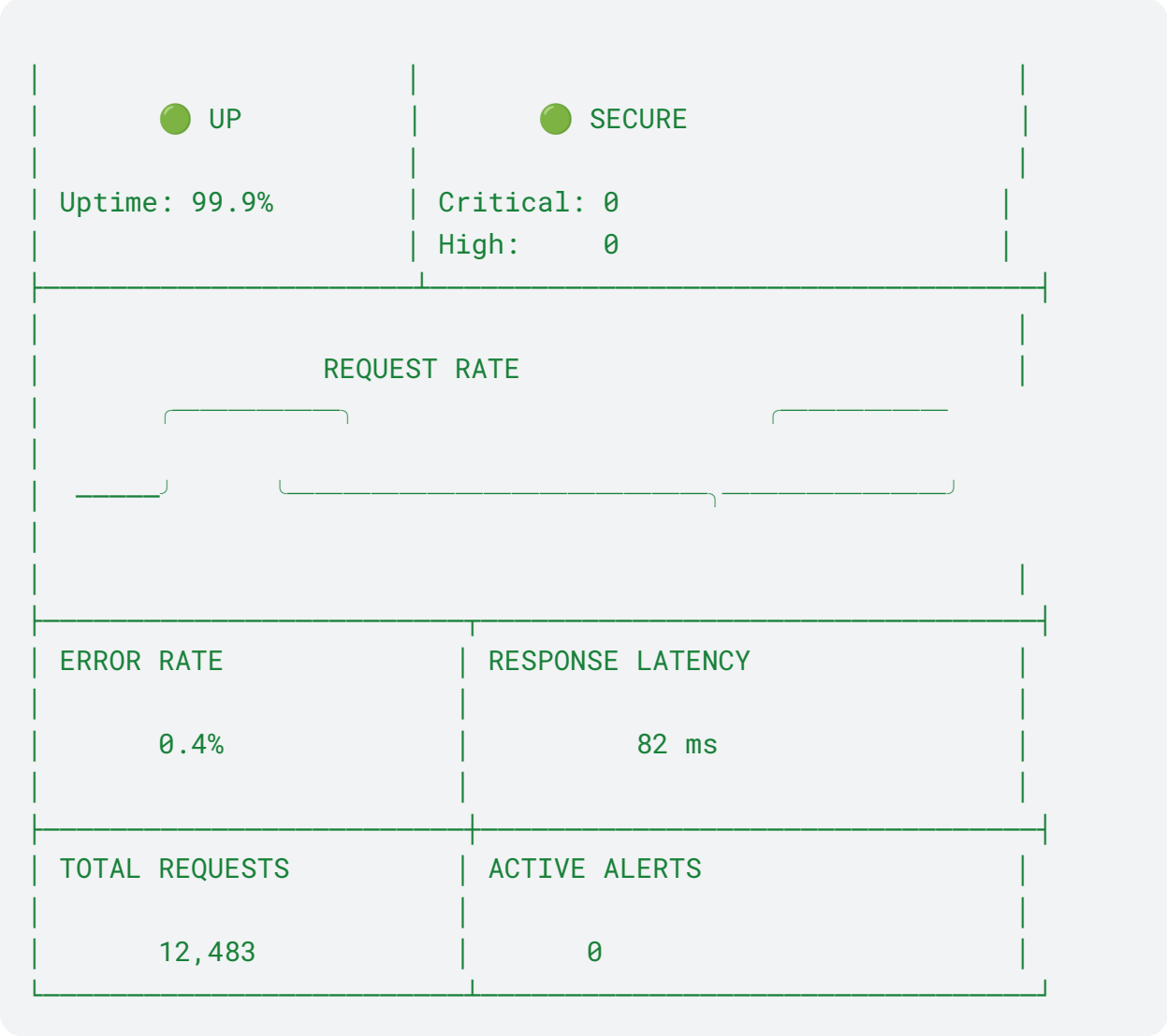
It should culminate in **one unified DevSecOps + Observability dashboard and live workflow**.

Imagine the evaluator sees this on your screen:

🖥️ SecureMonitor — Production Dashboard

None

```
┌──────────────────────────────────────────────────────────────┐
│              SECUREMONITOR DASHBOARD                         │
├──────────────────────────┬───────────────────────────────────┤
│ APPLICATION STATUS       | SECURITY STATUS                   │
```



That's your **final visual output**.

---

15. But your live demonstration should show THREE states

This is what I'd specifically engineer your presentation around.

STATE 1 — Healthy + Secure

```
None
Security        🟢 PASS
Application     🟢 UP
Error Rate      0.2%
Latency         80ms
Alerts          0
```

Say:

> "This is our normal production state."

STATE 2 — Security Failure

Introduce your controlled vulnerable code.

Run your security pipeline.

Display:

```
None
Security Scan

❌ Vulnerability Detected

Security Gate: FAILED

Deployment: BLOCKED
```

Then explain:

> "The application cannot be promoted because it has failed our security policy."

That's your **DevSecOps demonstration**.

STATE 3 — Runtime Failure

Fix the security issue and deploy.

Then generate excessive errors.

Your Grafana dashboard changes:

```
Application     🔴 DEGRADED
Error Rate      19.4%
Latency         1.2 sec
Alert           🚨 HIGH ERROR RATE
```

Then stop the error generation.

Eventually:

```
Application     🟢 HEALTHY
Error Rate      0.3%
Latency         85 ms
Alert           ✓ RESOLVED
```

That's your **observability demonstration**.

---

16. Your complete project phases

I'd structure your actual development like this:

Phase	What you implement	Output
1. Application	Build Flask application	Working web app
2. Instrumentation	Add application metrics	`/metrics` endpoint
3. Prometheus	Collect/store metrics	Prometheus targets
4. Grafana	Connect Prometheus	Live dashboard

5. Security	Add CodeQL/security scanning	Security findings
6. Web Security	Add OWASP ZAP	Web vulnerability report
7. Security Gate	Block unsafe deployment	PASS/FAIL pipeline
8. Alerts	Configure monitoring thresholds	Runtime alerts
9. Testing	Generate traffic/errors	Changing metrics
10. Integration	Connect everything	Complete DevSecOps system
11. Demo Scenarios	Healthy → security failure → runtime failure → recovery	Live demonstration
12. Documentation	Screenshots, architecture, results	Final report/presentation

---

17. What you should NOT add

This is important.

Don't turn this into:

```
None
Git
Docker
Kubernetes
Jenkins
Terraform
Ansible
Prometheus
Grafana
CodeQL
ZAP
...
```

You'll end up with a giant project that doesn't have a clear concept.

Instead:

Your two primary concepts are:

1. DevSecOps

Security integrated into the development/deployment lifecycle.

2. Observability

Using metrics, dashboards and alerts to understand application health.

Everything else is a supporting technology.

---

18. Suggested technology stack

I'd keep it relatively lightweight:

```
None
Application:
Python + Flask

Source:
Git + GitHub

Security:
CodeQL
OWASP ZAP

Monitoring:
Prometheus

Visualization:
Grafana

Automation:
GitHub Actions
```

CodeQL is particularly useful because it can be integrated into GitHub Actions and produce code-scanning alerts. (GitHub Docs)

For web security, ZAP's baseline scan is well suited to an automated demonstration because it performs passive scanning rather than launching a full active attack. (ZAP)

For monitoring, Prometheus → Grafana is a natural pairing: Prometheus provides the time-series metrics, while Grafana queries and visualizes them. (Prometheus)

🏆 Why I think this can score extremely well

Your rubric effectively asks:

Conceptual Understanding — 5

You can explain **two modern DevOps ideas** and, importantly, how they complement each other.

```
None
DevSecOps = Prevent problems before deployment

Observability = Detect problems after deployment
```

That is an excellent conceptual story.

Implementation — 6

You're not showing screenshots of tools.

You actually have:

```
None
Application
+
Security scanning
+
Security gate
+
```

```
Metrics
+
Prometheus
+
Grafana
+
Alerts
```

Demonstration — 4

You have a rehearsed sequence:

```
None
Healthy
 ↓
Security vulnerability
 ↓
Pipeline blocks deployment
 ↓
Fix
 ↓
Deploy
 ↓
Generate runtime errors
 ↓
Grafana detects problem
 ↓
Alert
 ↓
Recovery
```

That's an **extremely strong live demo**.

Documentation — 5

Your report can contain:

1. Problem statement
2. Objectives
3. DevSecOps concept
4. Observability concept
5. Architecture
6. Technology stack
7. Application implementation
8. Security implementation
9. Monitoring implementation
10. Dashboard
11. Security failure scenario
12. Monitoring failure scenario
13. Recovery
14. Results
15. Real-world applications
16. Limitations
17. Conclusion

One modification I'd make to your original idea

I would **not title it simply**:

> "Prometheus + Grafana and DevSecOps"

That sounds like two unrelated lab demonstrations.

Instead, make the central theme:

> ### "Secure and Observable Application Delivery: An Integrated DevSecOps and Monitoring Framework"

And your central research/problem statement:

> **"How can security vulnerabilities be detected before deployment and application failures be detected after deployment using an automated DevOps workflow?"**

Then your entire project answers that one question.

That is a **much more mature IA-II project** and, importantly, it is clearly differentiated from your listed Lab 1–9 experiments.